AI
AI AGENTS
EVERYWHERE!

Agents Definition

Agent is: an AI model capable of reasoning, planning, and interacting with its environment.

A More precise Definition

An Agent is a system that leverages an AI model to interact with its environment in order to achieve a user-defined objective. It combines reasoning, planning, and the execution of actions (often via external tools) to fulfill tasks

Reference: AI Engineering by Chip Huyen

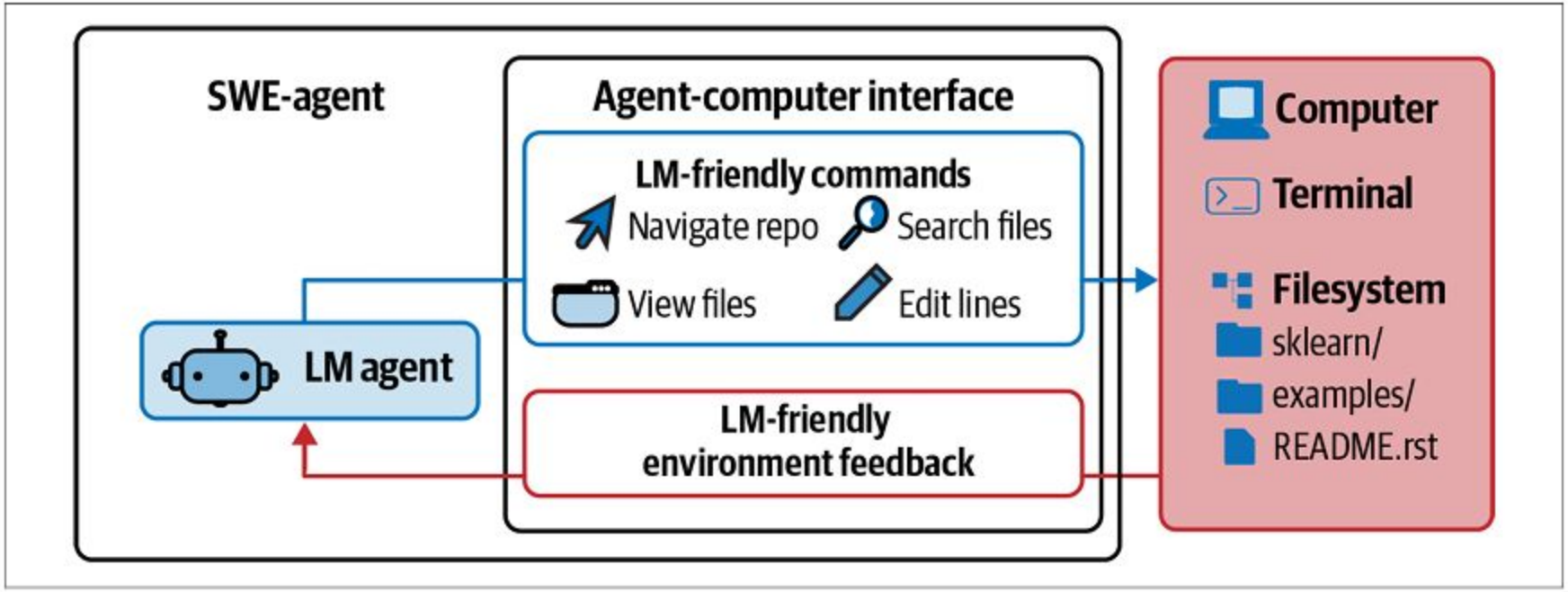


Figure 6-8. SWE-agent (Yang et al., 2024) is a coding agent whose environment is the computer and whose actions include navigation, search, and editing. Adapted from an original image licensed under CC BY 4.0.

Reference: AI Engineering by Chip Huyen

Anatomy of an Agent

Think of the Agent as having two main parts:

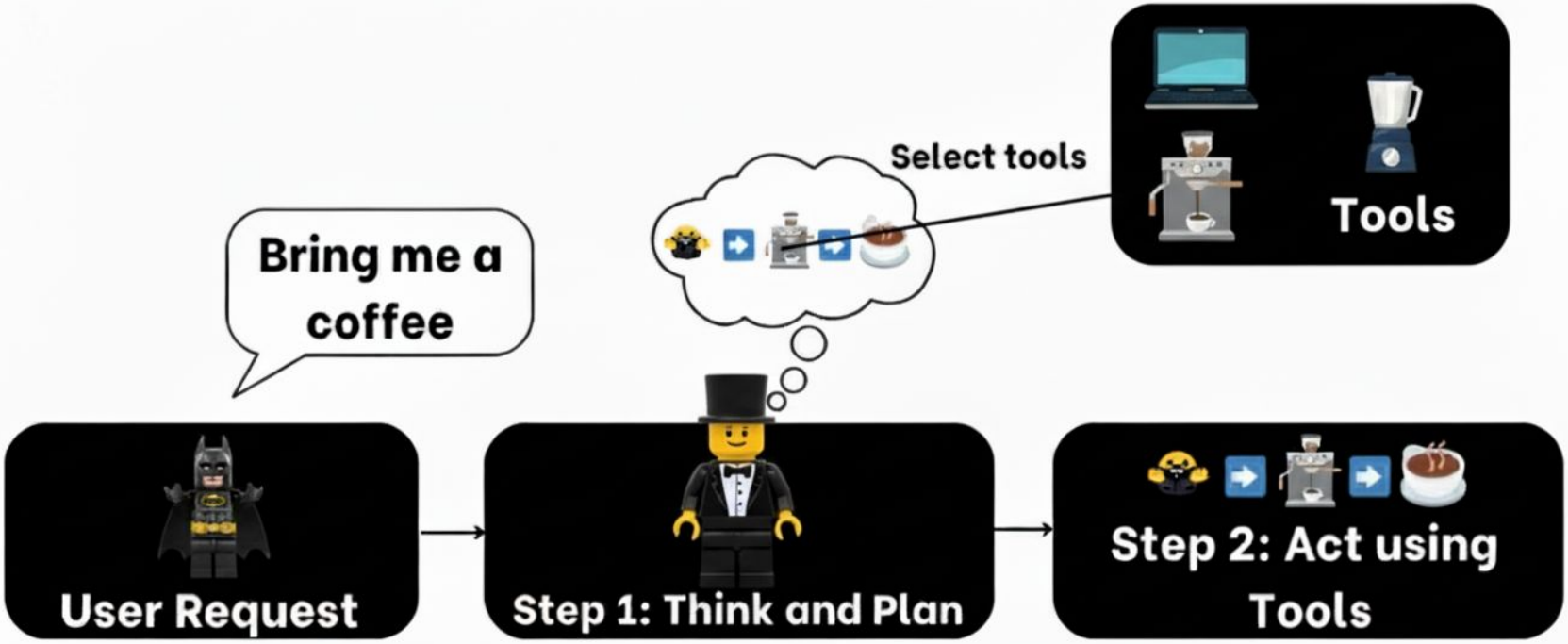
1. **The Brain (AI Model)**

This is where all the thinking happens. The AI model **handles reasoning and planning**. It decides **which Actions to take based on the situation**.

2. **The Body (Capabilities and Tools)**

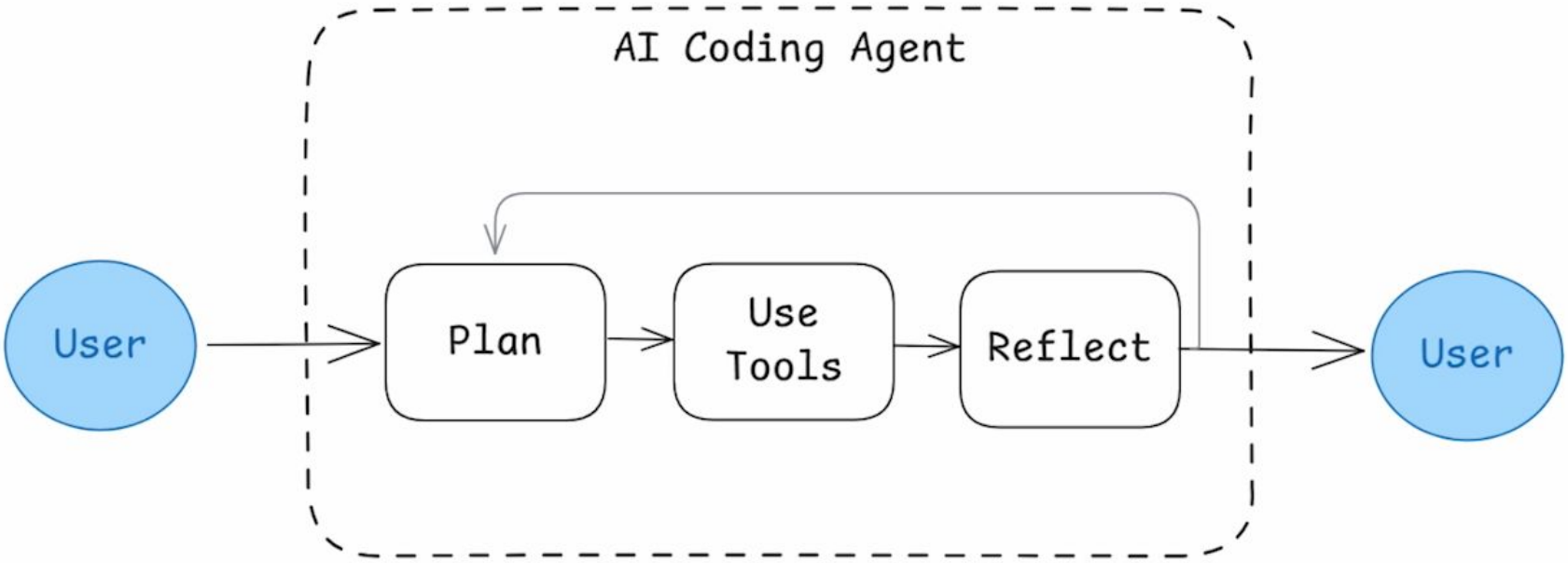
This part represents **everything the Agent is equipped to do**.

The **scope of possible actions** depends on what the agent **has been equipped with**. For example, because humans lack wings, they can't perform the "fly" **Action**, but they can execute **Actions** like "walk", "run" ,"jump", "grab", and so on.



We call it Agent because it has agency, aka it has the ability to interact with the environment.

AI Coding Agent



The spectrum of “Agency”

Following this definition, Agents exist on a continuous spectrum of increasing agency:

Agency Level	Description	What that’s called	Example pattern
☆☆☆	Agent output has no impact on program flow	Simple processor	`process_llm_output(llm_response)`
★☆☆	Agent output determines basic control flow	Router	`if llm_decision(): path_a() else: path_b()`
★★☆	Agent output determines function execution	Tool caller	`run_function(llm_chosen_tool, llm_chosen_args)`
★★★	Agent output controls iteration and program continuation	Multi-step Agent	`while llm_should_continue(): execute_next_step()`
★★★	One agentic workflow can start another agentic workflow	Multi-Agent	`if llm_trigger(): execute_agent()`

What’s the current weather in Paris?

The LLM hallucinates

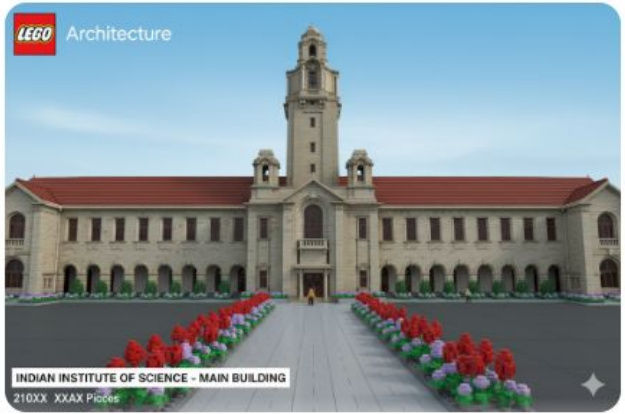
Agent that has WebSearch Tool

What type of AI Models do we use for Agents ?

1. LLM Text In —> Text Out
2. VLM Text + Image In → Text Out

Image Generation as an External tool to Gemini

Create a photorealistic LEGO model of the IISc Main Building in the style of an official LEGO box art



Where should we begin?

Ask anything

- Add photos & files
- Deep research
- Create image
- Think longer
- Study and learn
- More
  - Web search
  - Canvas
  - Connect Google Drive
  - Connect OneDrive
  - Connect Sharepoint

System Prompt

```
system_message="""You are an AI assistant designed to help users efficiently and accurately. Your
primary goal is to provide helpful, precise, and clear responses.

You have access to the following tools:
{tools_description}
"""
```

System Prompt with Tool Calling

```
system_message="""You are an AI assistant designed to help users efficiently and accurately. Your
primary goal is to provide helpful, precise, and clear responses.

You have access to the following tools:
Tool Name: calculator, Description: Multiply two integers., Arguments: a: int, b: int, Outputs: int
"""
```

```
system_message="""You are an AI assistant designed to help users efficiently and accurately. Your
primary goal is to provide helpful, precise, and clear responses.

You have access to the following tools:
Tool Name: calculator, Description: Multiply two integers., Arguments: a: int, b: int, Outputs: int

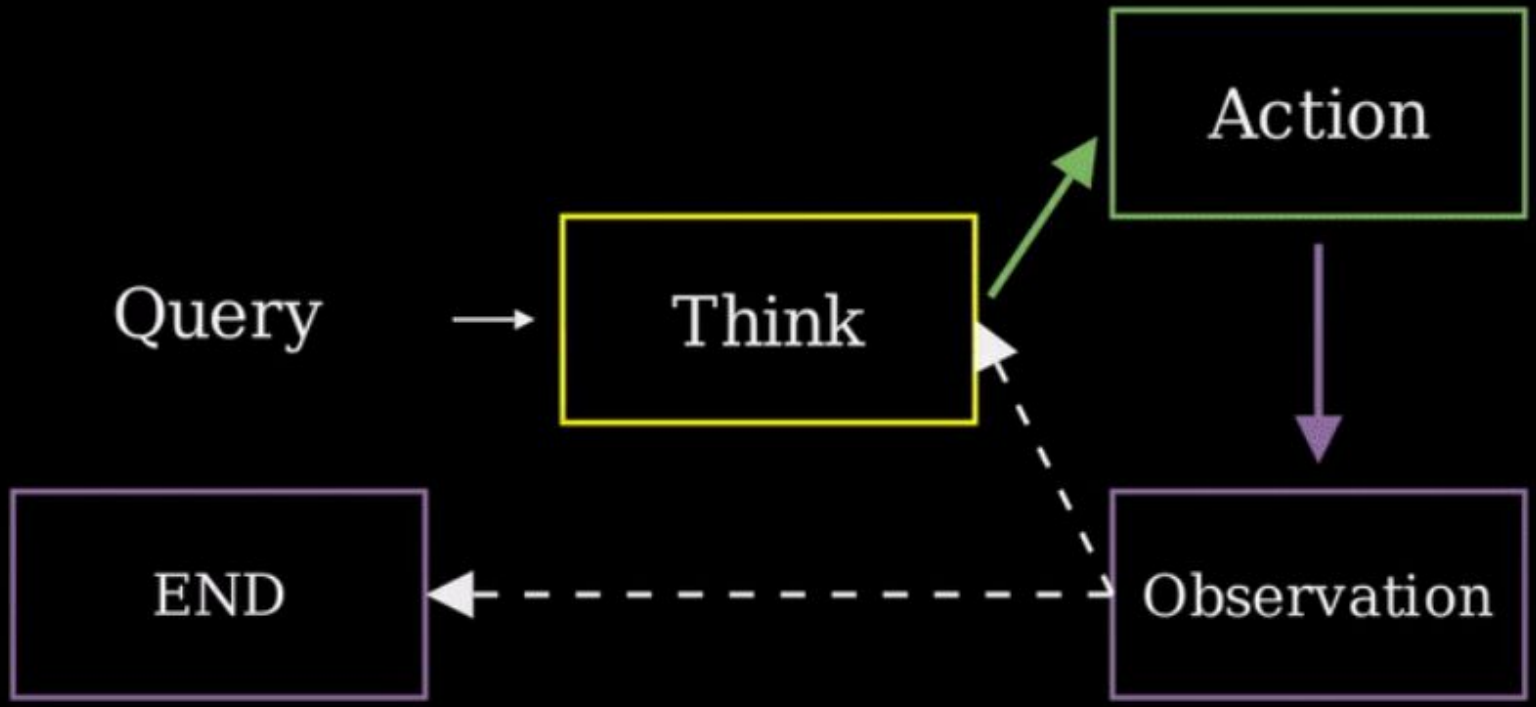
You should think step by step in order to fulfill the objective with a reasoning divided into
Thought/Action/Observation steps that can be repeated multiple times if needed.

You should first reflect on the current situation using `Thought: {your_thoughts}`, then (if
necessary), call a tool with the proper JSON formatting `Action: {JSON_BLOB}`, or print your final
answer starting with the prefix `Final Answer:`
"""
```

In many Agent frameworks, the rules and guidelines are embedded directly into the system prompt, ensuring that every cycle adheres to a defined logic.

In a simplified version, our system prompt may look like this:

LLM Thought Process



Actions: Enabling the Agent to Engage with Its Environment

Whether it’s browsing the web for information or controlling a physical device, each action is a deliberate operation executed by the agent.

How to Implement Actions ?

The Stop and Parse Approach

One key method for implementing actions is the **stop and parse approach**. This method ensures that the agent's output is structured and predictable:

1. **Generation in a Structured Format**:

The agent outputs its intended action in a clear, predetermined format (JSON or code).

2. **Halting Further Generation**:

Once the text defining the action has been emitted, **the LLM stops generating additional tokens**. This prevents extra or erroneous output.

3. **Parsing the Output**:

An external parser reads the formatted action, determines which Tool to call, and extracts the required parameters.

For example, an agent needing to check the weather might output:

Actions: Enabling the Agent to Engage with Its Environment

Types of Agent Actions

There are multiple types of Agents that take actions differently:

Type of Agent	Description
JSON Agent	The Action to take is specified in JSON format.
Code Agent	The Agent writes a code block that is interpreted externally.
Function-calling Agent	It is a subcategory of the JSON Agent which has been fine-tuned to generate a new message for each action.

Code Agents

Code Agents are agents that write code to perform a sequence of actions.
The Idea behind this is to let your LLM write code for your agent as well to get stuff done.
Code Agents lets you take advantage of LLMs coding ability to write code to take actions

This is different from coding agents which are agents like those in Windsurf or Cursor that write code for us to execute

The Standard ReAct Loop

📋 Task: “Give me the weather in Stanford”

Step 1	Thought	*I need to call the weather API*
	▼	
	Action	*Call tool “weather_api” with arg “Stanford”*
	▼	
	⏳👀 Observation	*⇒ Output: “It’s sunny out there”*
	▼	
Step 2	Thought	*I can now return the answer*
	▼	
	Action	*Call tool “final_answer” with arg “sunny”*
	▼	
	⏳👀 Observation	*⇒ Output: “sunny”*
	▼	

✅ Agent output: “sunny”

How to Write an Action?

📋 Task: "Give me the weather in Stanford"

Step 1

Thought — *I need to call the weather API*

▼

Action — *Call tool "weather_api" with arg "Stanford"*

▼

⏳👀 Observation — *⇒ Output: "It's sunny out there"*

How to write this action?

How to Write an Action?

📋 Task: "Give me the weather in Stanford"

Step 1

Thought — *I need to call the weather API*

▼

Action — *Call tool "weather_api" with arg "Stanford"*

▼

⏳👀 Observation — *⇒ Output: "It's sunny out there"*

How to write this action?

JSON/text action:

```
I need to call the weather API
{"name": "weather_api", "arg": "Stanford"}
```

How to Write an Action?

📋 Task: "Give me the weather in Stanford"

Step 1

Thought — *I need to call the weather API*

▼

Action — *Call tool "weather_api" with arg "Stanford"*

▼

⏳👀 Observation — *⇒ Output: "It's sunny out there"*

How to write this action?

JSON/text action:

```
I need to call the weather API
{"name": "weather_api", "arg": "Stanford"}
```

Code action:

```
I need to call the weather API
```py
weather_api("Stanford")
```
```

How to Write an Action?

📋 Task: "Give me the weather in Stanford"

Step 1

Thought — *I need to call the weather API*

▼

Action — *Call tool "weather_api" with arg "Stanford"*

▼

⏳👀 Observation — *⇒ Output: "It's sunny out there"*

How to write this action?

JSON/text action:

```
I need to call the weather API
{"name": "weather_api", "arg": "Stanford"}
```

Code action:

```
I need to call the weather API
```py
weather_api("Stanford")
```
```

The Unfair Advantage of Code

📋 **Task:** "Determine the most cost-effective country to purchase a smartphone: the USA, Japan, Germany, or India?"

🛠️ **Tools:** get_rate, get_phone_price, convert_and_tax

The Unfair Advantage of Code

Code agent's solution:

```
I need to get final prices for each country and compare them.
```py
prices = {}
for country in ["USA", "Japan", "Germany", "India"]:
    price = get_phone_price(country)
    exchange_rate = get_rate(country)
    prices[country] = convert_and_tax(price, country, exchange_rate)

best_country = min(prices, key=prices.get)
final_answer(f"Best country: {best_country}, Price: ${prices[best_country]}")
```
```

JSON/Text agent's solution: 4 countries × 3 calls = **12 steps**

```
I need to get the phone price for Germany.
I need to get the exchange rate for Germany.
I need to convert and apply taxes for Germany.
{"name": "convert_and_tax", "arg": "Germany"}
I need to get the exchange rate for India.
I need to convert and apply taxes for India.
{"name": "convert_and_tax", "arg": "India"}
I need to get the phone price for Japan.
I need to get the exchange rate for Japan.
I need to convert and apply taxes for Japan.
{"name": "convert_and_tax", "arg": "Japan"}
```

Source: *Executable Code Actions Elicit Better LLM Agents*

Code Agents Perform Better

Executable Code Actions Elicit Better LLM Agents

Xingyao Wang [1] Yangyi Chen [1] Lifan Yuan [1] Yizhe Zhang [2] Yunzhu Li [1] Hao Peng [1] Heng Ji [1]

Figure 1: Comparison between `CodeAct` and Text / JSON as action. **(top)** Illustrative example comparing different actions. **(bottom)** Quantitative results on M^3ToolEval (§2.3).

This approach offers several advantages:

- **Expressiveness:** Code can naturally represent complex logic, including loops, conditionals, and nested functions, providing greater flexibility than JSON.
- **Modularity and Reusability:** Generated code can include functions and modules that are reusable across different actions or tasks.
- **Enhanced Debuggability:** With a well-defined programming syntax, code errors are often easier to detect and correct.
- **Direct Integration:** Code Agents can integrate directly with external libraries and APIs, enabling more complex operations such as data processing or real-time decision making.

The Frameworks handle the execution

Framework	Built-in tool catalog	Examples of tools	Can you add your own?
LangChain / LangGraph	Hundreds of community and native integrations `python.langchain` +1	Web search, SQL, Python REPL, browser automation, vector DB queries	Yes—inherit `BaseTool` or just expose any Python function
LlamaIndex	RAG-oriented loaders, query engines, DB connectors `developers.llamaindex`	PDF loader, SQL query engine, vector search, Text-to-SQL	Yes—wrap any data loader or function as a `ToolSpec`
smolagents	Lean core; focuses on executing arbitrary Python but still ships helper utilities `huggingface` +1	`browser_search`, `pip_install`, `python exec` sandbox	Yes—since agents can import any whitelisted package or you can register helper functions
n8n	400+ drag-and-drop nodes (HTTP, Slack, GitHub, databases, spreadsheets, etc.) `n8n` +1	"HTTP Request," "MySQL," "Python," "OpenAI," "Send Email"	Yes—create custom nodes or run inline JavaScript/Python
CrewAI	Library of file, web-scraping, DB, vector-DB, and AI tools `github` +1	`FileReadTool`, `ScrapeWebsiteTool`, `PGSearchTool`, `DallETool`	Yes—implement the simple `BaseTool` interface

Observe: Integrating Feedback to Reflect and Adapt

Observations are **how an Agent perceives the consequences of its actions**.

They provide crucial information that fuels the Agent's thought process and guides future actions.

They are **signals from the environment**—whether it's data from an API, error messages, or system logs—that guide the next cycle of thought.

In the observation phase, the agent:

- **Collects Feedback:** Receives data or confirmation that its action was successful (or not).
- **Appends Results:** Integrates the new information into its existing context, effectively updating its memory.
- **Adapts its Strategy:** Uses this updated context to refine subsequent thoughts and actions.

For example, if a weather API returns the data *"partly cloudy, 15°C, 60% humidity"*, this observation is appended to the agent's memory (at the end of the prompt).

The Agent then uses it to decide whether additional information is needed or if it's ready to provide a final answer.

This **iterative incorporation of feedback ensures the agent remains dynamically aligned with its goals**, constantly learning and adjusting based on real-world outcomes.

These observations **can take many forms**, from reading webpage text to monitoring a robot arm's position. This can be seen like Tool "logs" that provide textual feedback of the Action execution.

Type of Observation	Example
System Feedback	Error messages, success notifications, status codes
Data Changes	Database updates, file system modifications, state changes
Environmental Data	Sensor readings, system metrics, resource usage
Response Analysis	API responses, query results, computation outputs
Time-based Events	Deadlines reached, scheduled tasks completed

How Are the Results Appended?

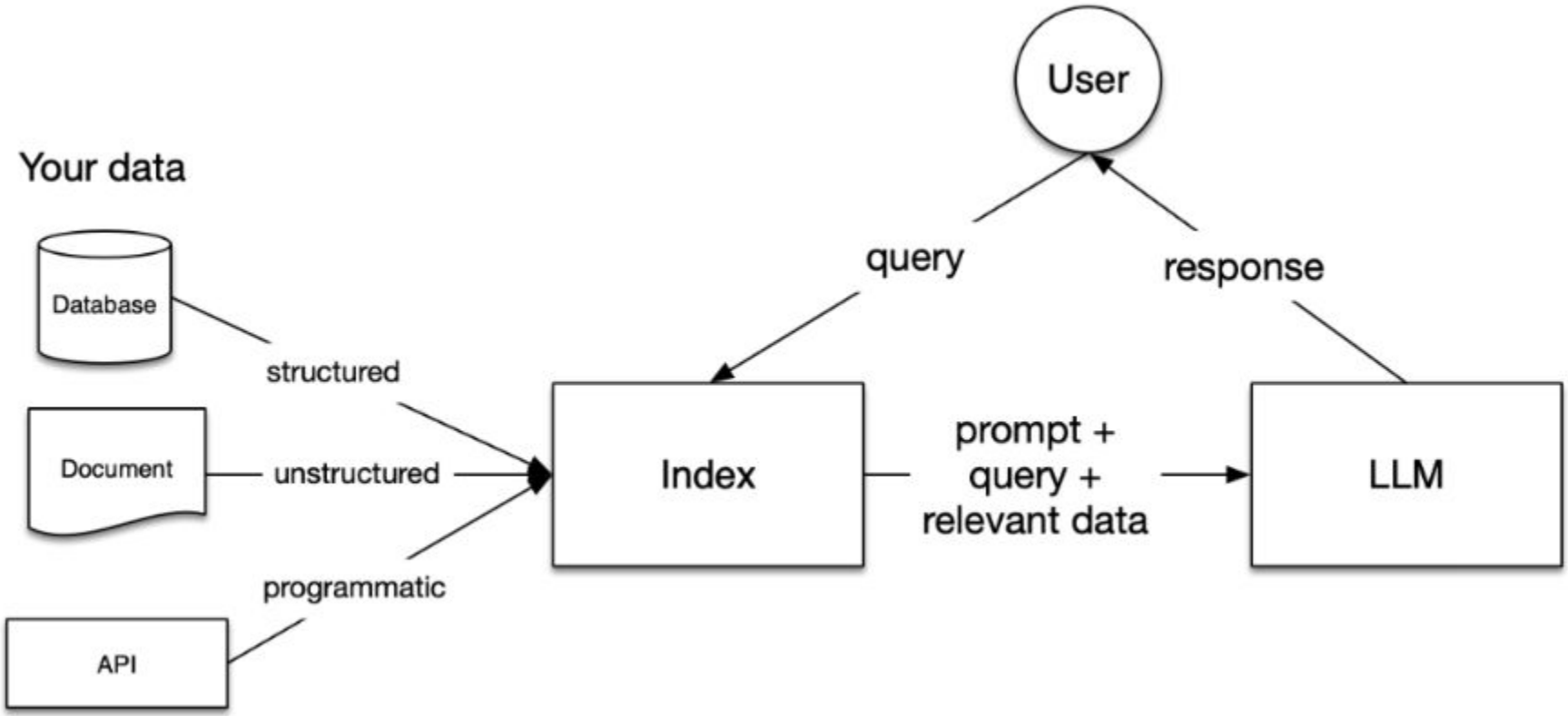
After performing an action, the framework follows these steps in order:

1. **Parse the action** to identify the function(s) to call and the argument(s) to use.
2. **Execute the action.**
3. **Append the result** as an **Observation**.

How do Multi-Step Agents work ?

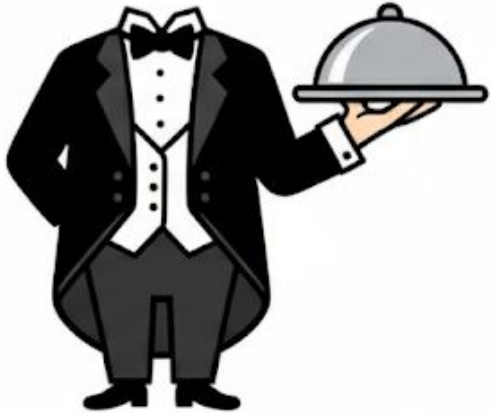
https://cas-bridge.xethub.hf.co/xet-bridge-us/621ffdd236468d709f1835cf/00598243d02aefce27b6f2a315b745b55556b191fd8f472a95d3c0f2695ed84d?X-Amz-Algorithm=AWS4-HMAC-SHA256&X-Amz-Content-Sha256=UNSIGNED-PAYLOAD&X-Amz-Credential=cas%2F20250928%2Fus-east-1%2Fs3%2Faws4_request&X-Amz-Date=20250928T200126Z&X-Amz-Expires=3600&X-Amz-Signature=554015ee20ac5e46122bec81e223258d36a1b9cd0fd7a25bd991ce700bebdd5b&X-Amz-SignedHeaders=host&X-Xet-Cas-Uid=68d5541d0f87577c00cecad4&response-content-disposition=inline%3B+filename*%3DUTF-8%27%27Agent_ManimCE.gif%3B+filename%3D%22Agent_ManimCE.gif%22%3B&response-content-type=image%2Fgif&x-id=GetObject&Expires=1759093286&Policy=eyJTdGF0ZW1lbnQiOlt7IkNvbmRpdGlvbiI6eyJEYXRlTGVzc1RoYW4iOnsiQVdTOkVwb2NoVGltZSI6MTc1OTA5MzI4Nn19LCJSZXNvdXJjZSI6Imh0dHBzOi8vY2FzLWJyaWRnZS54ZXRodWIuaGYuY28veGV0LWJyaWRnZS11cy82MjFmZmRkMjM2NDY4ZDcwOWYxODM1Y2YvMDA1OTgyNDNkMDJhZWZjZTI3YjZmMmEzMTViNzQ1YjU1NTU2YjE5MWZkOGY0NzJhOTVkM2MwZjI2OTVlZDg0ZCoifV19&Signature=I6eH6vtaibdoh8UaoqnkdPewpDaWA5B-kNAQd386YML6xcY%7E5BvQ1IJ0NglrSXi4Jqp9uE3Vk8p%7EiDiiBNHl5ayVnVlnr2mqE1Ie5nbz%7Eg%7ENqfGAd9j%7E5XW3ZCUd%7E3H010zfxaMzqItON3I1K8t5mvmuq0ebewjVLTSqn5SFZJiRPFaWQmAPbjcvryRWODJOMKeqH%7EUMZBY2rnNRtBjhXolUcNHGU1cHUzaN5fa9SW5akrO9sP%7EiEMpAEqhthwFSnwnHWXxA8z7q%7ENUCQqJ0KRNWd86vz5Jirl2OSUMBKCgorYhU4jUd6AWJsrzogtLpGy-EOSkGocbRphImsGlkWg__&Key-Pair-Id=K2L8F4GPSG1IFC

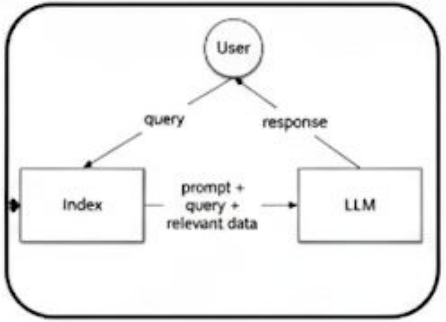
Normal RAG


User
Your data
query
response
Database
structured
Document
unstructured
Index
prompt +
query +
relevant data
LLM
programmatic
API

Agentic RAG

I can do Agentic RAG if you provide me with a RAG pipeline tool and some other tools I could use to respond directly





External API

Web Search

Direct LLM Response

Agent Evaluation Benchmark

GAIA tasks are organized into three levels of increasing complexity, each testing specific skills:

- Level 1: Requires less than 5 steps and minimal tool usage.
- Level 2: Involves more complex reasoning and coordination between multiple tools and 5-10 steps.
- Level 3: Demands long-term planning and advanced integration of various tools.

Level 1

Question: What was the actual enrollment count of the clinical trial on H. pylori in acne vulgaris patients from Jan-May 2018 as listed on the NIH website?
Ground truth: 90

Level 2

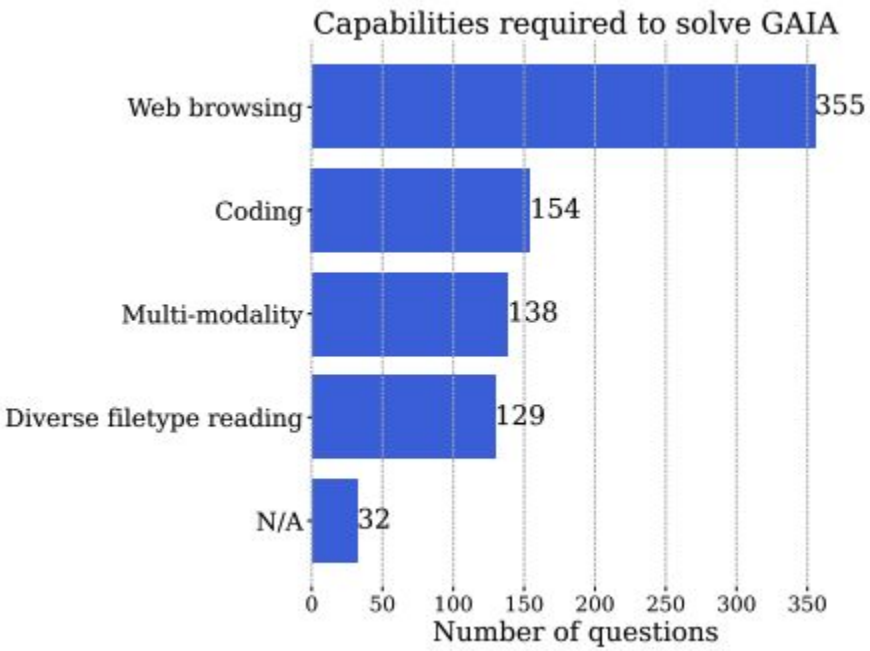
Question: If this whole pint is made up of ice cream, how many percent above or below the US federal standards for butterfat content is it when using the standards as reported by Wikipedia in 2020? Answer as + or - a number rounded to one decimal place.
Ground truth: +4.6

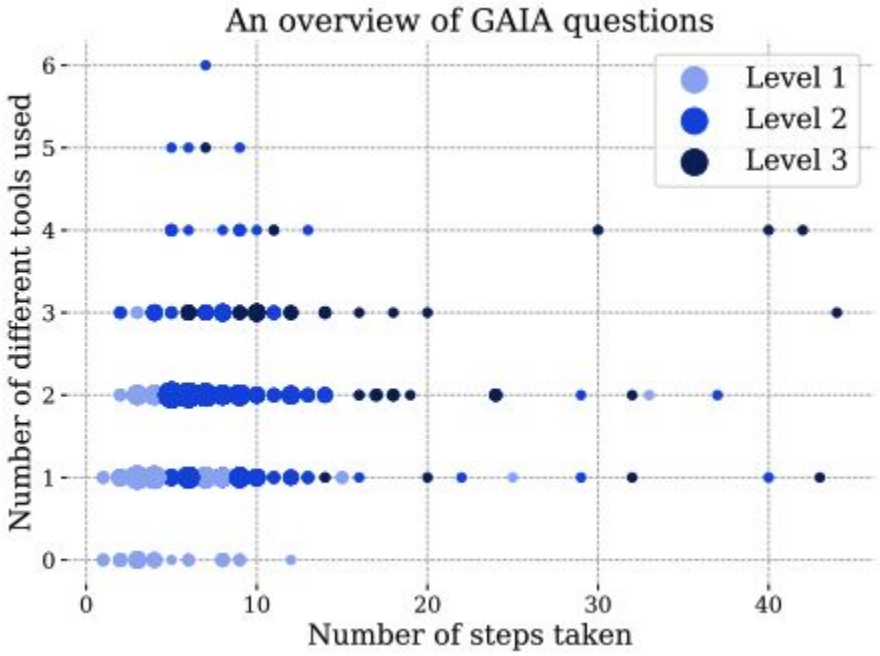
Level 3

Question: In NASA's Astronomy Picture of the Day on 2006 January 21, two astronauts are visible, with one appearing much smaller than the other. As of August 2023, out of the astronauts in the NASA Astronaut Group that the smaller astronaut was a member of, which one spent the least time in space, and how many minutes did he spend in space, rounded to the nearest minute? Exclude any astronauts who did not spend any time in space. Give the last name of the astronaut, separated from the number of minutes by a semicolon.
Ground truth: White; 5876

Example of a Hard GAIA Question

Which of the fruits shown in the 2008 painting “Embroidery from Uzbekistan” were served as part of the October 1949 breakfast menu for the ocean liner that was later used as a floating prop for the film “The Last Voyage”? Give the items as a comma-separated list, ordering them in clockwise order based on their arrangement in the painting starting from the 12 o’clock position. Use the plural form of each fruit.


Capabilities required to solve GAIA
Web browsing
Coding
Multi-modality
Diverse filetype reading
N/A
355
154
138
129
32
0
50
100
150
200
250
300
350
Number of questions


An overview of GAIA questions
Level 1
Level 2
Level 3
Number of different tools used
0
1
2
3
4
5
6
0
10
20
30
40
Number of steps taken